Name: Ryaan Devnani	Class: T1	Roll No: 2401023
Batch: T11	Subject: Agile Software Development & DevOps	Deptartment: AI & DS

Experiment no. 03

Aim: To deploy multi-service application using Docker compose

LO4: To automate provisioning, configuration, container orchestration, and cloud resource management using Ansible, Docker, Kubernetes, and Terraform.

Theory:

Docker Compose is a tool provided by Docker that is used to define and run multi-container applications using a single YAML configuration file (docker-compose.yml). Instead of creating and starting each container separately using individual docker run commands, Docker Compose allows all the services required by an application — such as a web server, a database, and supporting tools — to be defined together in one file, and started, stopped, or managed together using a single command such as docker compose up. This directly supports the aim of this experiment: deploying a multi-service application as one unit rather than as separate, manually managed containers.

In real-world applications, a single container is rarely sufficient — a typical application may require a web server, a database, a caching layer, and other supporting services running together and communicating with each other. Managing each of these manually with individual docker run commands is tedious and error-prone.

Docker Compose solves this by allowing all services, along with their images, environment variables, ports, volumes, and dependencies, to be defined declaratively in the docker-compose.yml file. A single command, docker compose up, then reads this file and creates, configures, and starts all the defined services together, connected on a common Compose-managed network so they can communicate with each other using their service names as hostnames.

Key concepts used in this experiment:

Service: A single container definition within the compose file (e.g. web, database, phpmyadmin). Each service specifies the image to use, ports to expose, environment variables, and other configuration.

Image: The base image each service is built from — nginx for the web server, mysql:8.0 for the database, and phpmyadmin/phpmyadmin for the database admin UI.

Ports mapping ("host:container"): Maps a port on the host machine to a port inside the container, e.g. "8080:80" exposes the container's port 80 on the host's port 8080.

Environment variables: Used to configure a service at startup without modifying the image itself — here they set the MySQL root password, database name, and user credentials, and tell phpMyAdmin which database host to connect to.

Volumes: Named, persistent storage (mysql-data) that keeps database data intact even if the container is stopped, removed, or recreated.

depends_on: Specifies that one service depends on another, influencing the order in which containers are started (phpmyadmin is started after database).

This experiment uses three services instead of the minimum two, to demonstrate a more realistic multi-service setup: an nginx web server, a MySQL database, and a phpMyAdmin instance for managing that database through a browser-based interface — showing how Compose handles

inter-service communication (phpMyAdmin connecting to the database service purely by its service name, "database").

DOCKER COMPOSE FILE (docker-compose.yml)

The following configuration was created inside the compose-demo directory and saved as docker-compose.yml:

```
version: '3.8'

services:

  web:
    image: nginx
    container_name: webserver
    ports:
      - "8090:80"

  database:
    image: mysql:8.0
    container_name: mysqlserver
    environment:
      MYSQL_ROOT_PASSWORD: root123
      MYSQL_DATABASE: studentdb
      MYSQL_USER: student
      MYSQL_PASSWORD: student123
    ports:
      - "3307:3306"
    volumes:
      - mysql-data:/var/lib/mysql

  phpmyadmin:
    image: phpmyadmin/phpmyadmin
    container_name: phpmyadmin
    environment:
      PMA_HOST: database
      PMA_USER: student
      PMA_PASSWORD: student123
    ports:
      - "8081:80"
    depends_on:
      - database
```

```
volumes:
  mysql-data:
```

PROCEDURE

The following steps were carried out to set up and deploy the application:

#	Step / Command	Description
1	`mkdir compose-demo`	Creates a new project directory for this experiment.
2	`cd compose-demo`	Moves into the newly created directory.
3	`(create docker-compose.yml)`	The compose file is created using Notepad (saved as "All Files" so it is not saved with a .txt extension) inside the compose-demo folder.
4	`dir`	Lists the contents of the folder to confirm docker-compose.yml was saved correctly.
5	`docker compose up -d`	Reads docker-compose.yml and creates + starts all three services (webserver, mysqlserver, phpmyadmin) in detached (background) mode.

How the services connect

The web service (nginx) is accessible from the host browser at http://localhost:8090.

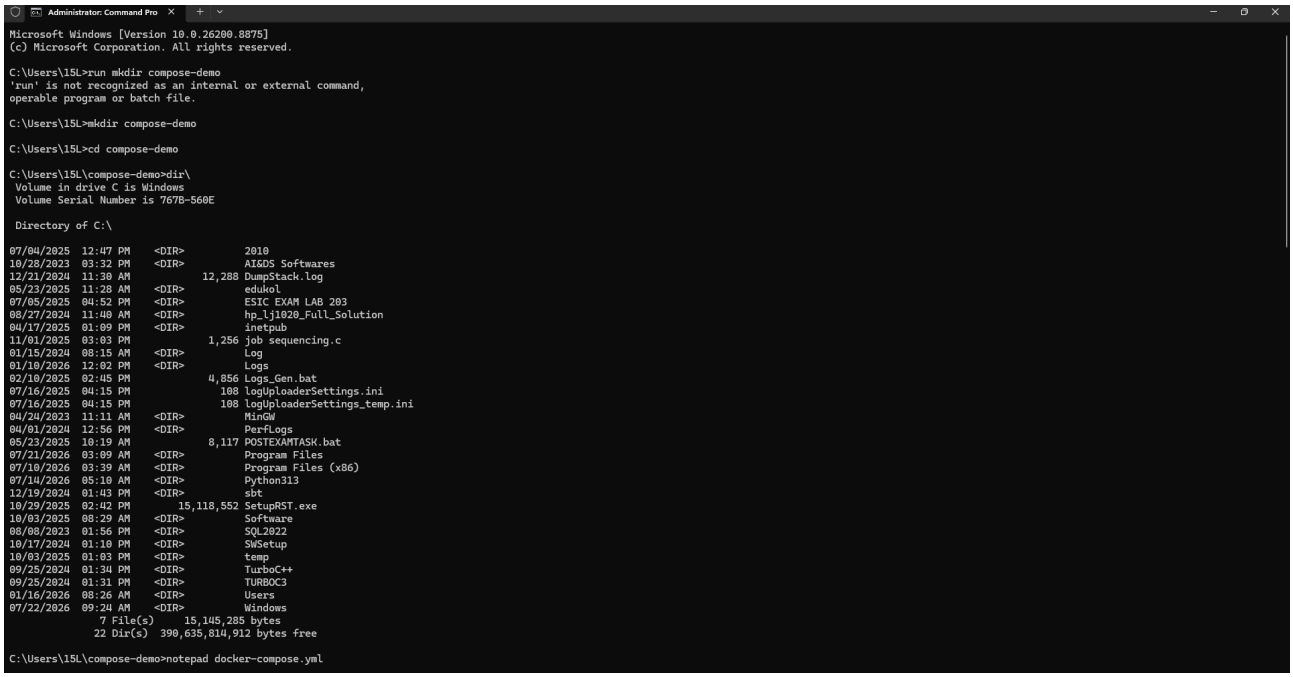
The database service (MySQL) is accessible on host port 3307, with credentials and database name set via environment variables, and its data persisted in the mysql-data volume.

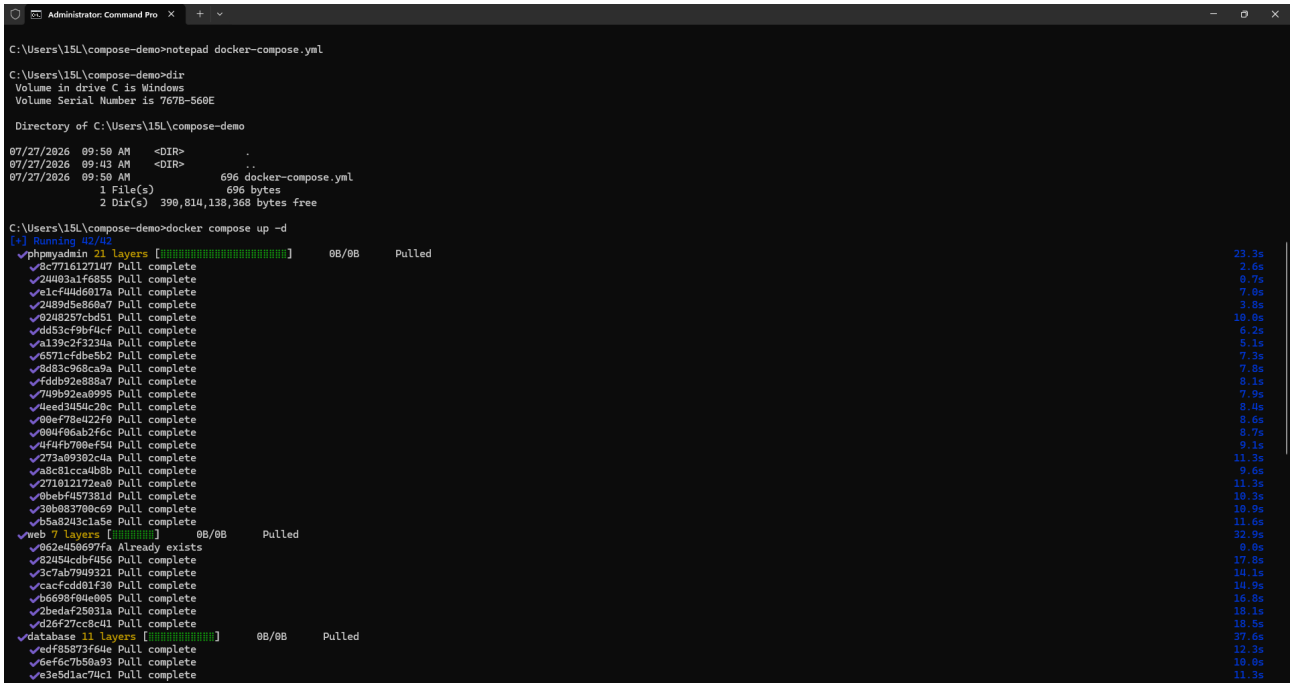
The phpmyadmin service is accessible at http://localhost:8081, and connects internally to the database service using the hostname "database" (its Compose service name) rather than an IP address — this works because Compose automatically places all services on the same internal network.

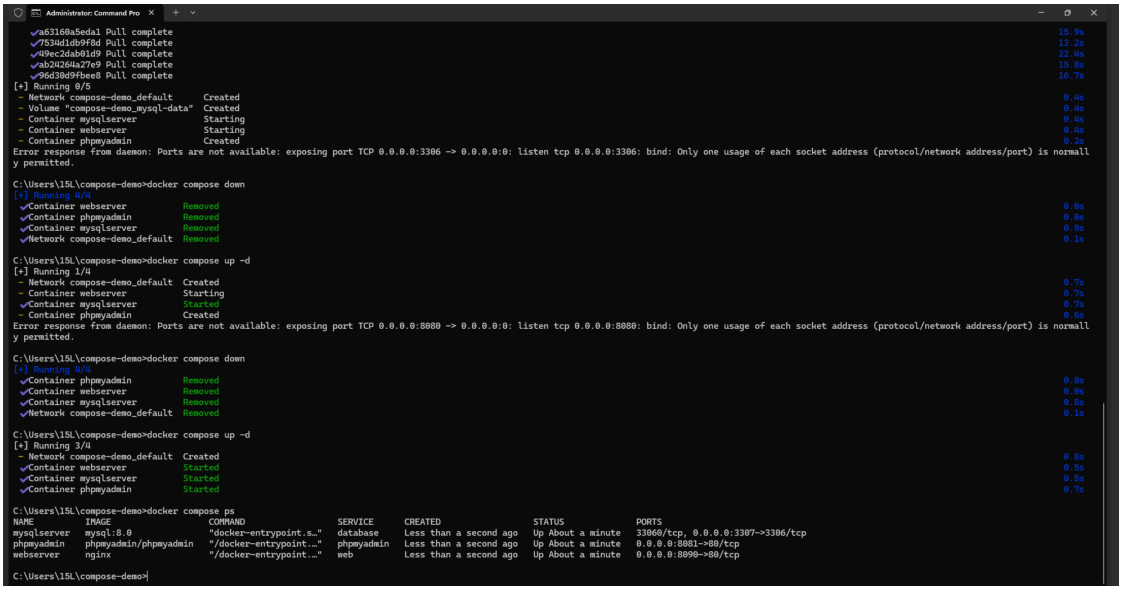
Note

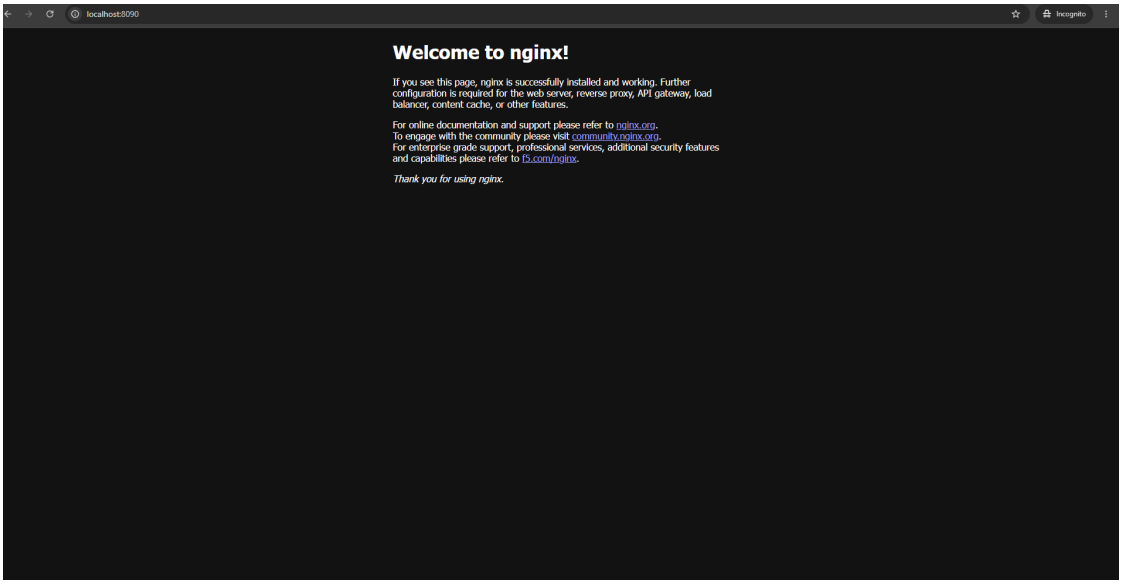
During deployment, docker compose up -d initially failed because the default host ports (3306 and 8080) specified in the file were already in use on the host machine. This was resolved by changing only the host-side port numbers to 3307 and 8090 respectively in docker-compose.yml, without altering the container-side ports, after which docker compose down followed by docker compose up -d started all three services successfully.

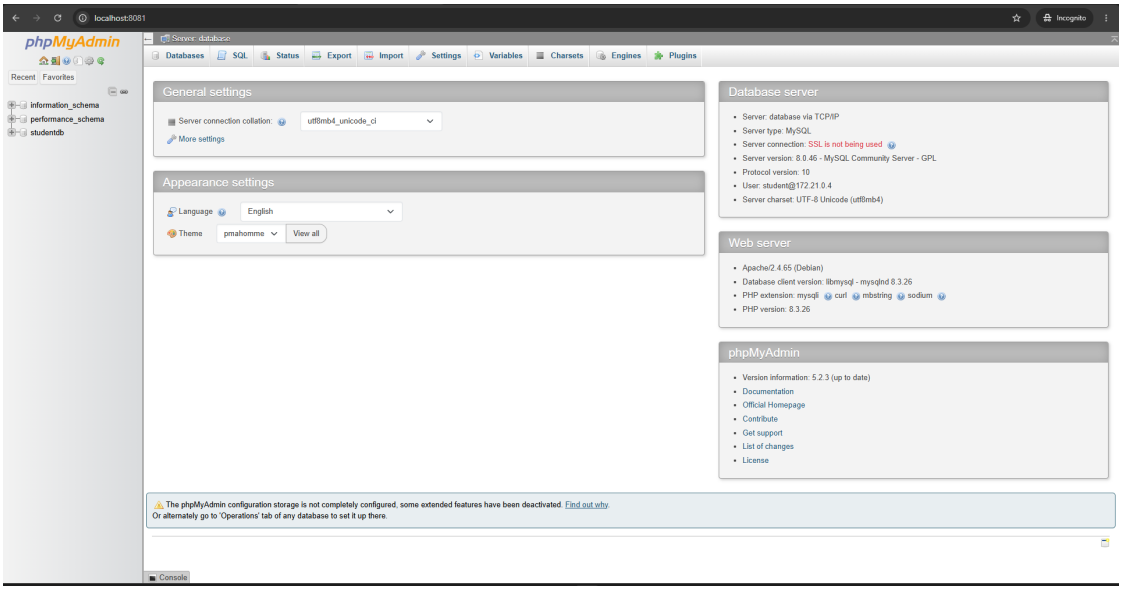
OBSERVATIONS / SCREENSHOTS






Administrator: Command Pro
a63160a5eda1 Pull complete 15.9s
7534d1db9f8d Pull complete 13.2s
49ec2dab01d9 Pull complete 22.4s
ab24264a27e9 Pull complete 15.8s
96d30d9fbee8 Pull complete 16.7s
[+] Running 0/5
- Network compose-demo_default Created 0.4s
- Volume "compose-demo_mysql-data" Created 0.4s
- Container mysqlserver Starting 0.4s
- Container webserver Starting 0.4s
- Container phpmyadmin Created 0.2s
Error response from daemon: Ports are not available: exposing port TCP 0.0.0.0:3306 -> 0.0.0.0:0: listen tcp 0.0.0.0:3306: bind: Only one usage of each socket address (protocol/network address/port) is normally permitted.
C:\Users\15L\compose-demo>docker compose down
[+] Running 4/4
Container webserver Removed 0.0s
Container phpmyadmin Removed 0.0s
Container mysqlserver Removed 0.0s
Network compose-demo_default Removed 0.1s
C:\Users\15L\compose-demo>docker compose up -d
[+] Running 1/4
- Network compose-demo_default Created 0.7s
- Container webserver Starting 0.7s
Container mysqlserver Started 0.7s
- Container phpmyadmin Created 0.6s
Error response from daemon: Ports are not available: exposing port TCP 0.0.0.0:8080 -> 0.0.0.0:0: listen tcp 0.0.0.0:8080: bind: Only one usage of each socket address (protocol/network address/port) is normally permitted.
C:\Users\15L\compose-demo>docker compose down
[+] Running 4/4
Container phpmyadmin Removed 0.0s
Container webserver Removed 0.0s
Container mysqlserver Removed 0.0s
Network compose-demo_default Removed 0.1s
C:\Users\15L\compose-demo>docker compose up -d
[+] Running 3/4
- Network compose-demo_default Created 0.8s
Container webserver Started 0.5s
Container mysqlserver Started 0.5s
Container phpmyadmin Started 0.7s
C:\Users\15L\compose-demo>docker compose ps
NAME IMAGE COMMAND SERVICE CREATED STATUS PORTS
mysqlserver mysql:8.0 "docker-entrypoint.s…" database Less than a second ago Up About a minute 33060/tcp, 0.0.0.0:3307->3306/tcp
phpmyadmin phpmyadmin/phpmyadmin "/docker-entrypoint.…" phpmyadmin Less than a second ago Up About a minute 0.0.0.0:8081->80/tcp
webserver nginx "/docker-entrypoint.…" web Less than a second ago Up About a minute 0.0.0.0:8090->80/tcp
C:\Users\15L\compose-demo>


localhost:8090
Incognito
Welcome to nginx!
If you see this page, nginx is successfully installed and working. Further configuration is required for the web server, reverse proxy, API gateway, load balancer, content cache, or other features.
For online documentation and support please refer to nginx.org.
To engage with the community please visit community.nginx.org.
For enterprise grade support, professional services, additional security features and capabilities please refer to f5.com/nginx.
Thank you for using nginx.


localhost:8081
Incognito
phpMyAdmin
Recent Favorites
information_schema
performance_schema
studentdb
Server: database
Databases SQL Status Export Import Settings Variables Charsets Engines Plugins
General settings
Server connection collation: utf8mb4_unicode_ci
More settings
Appearance settings
Language English
Theme pmahomme View all
Database server
Server: database via TCP/IP
Server type: MySQL
Server connection: SSL is not being used
Server version: 8.0.46 - MySQL Community Server - GPL
Protocol version: 10
User: student@172.21.0.4
Server charset: UTF-8 Unicode (utf8mb4)
Web server
Apache/2.4.65 (Debian)
Database client version: libmysql - mysqlnd 8.3.26
PHP extension: mysqli curl mbstring sodium
PHP version: 8.3.26
phpMyAdmin
Version information: 5.2.3 (up to date)
Documentation
Official Homepage
Contribute
Get support
List of changes
License
The phpMyAdmin configuration storage is not completely configured, some extended features have been deactivated. Find out why.
Or alternately go to 'Operations' tab of any database to set it up there.
Console

DOCKER COMPOSE COMMANDS USED

The following Docker Compose commands were used to manage the lifecycle of the multi-service application:

#	Command	Description
1	`docker compose up -d`	Starts all services defined in the compose file, in detached mode.
2	`docker compose ps`	Shows the status of all running services defined in the compose file.
3	`docker compose logs`	Displays combined logs from all services, useful for debugging.
4	`docker compose images`	Lists the images used by each service in the compose file.
5	`docker compose config`	Validates the syntax of the compose file and displays the fully resolved configuration.
6	`docker compose stop`	Stops all running services without removing the containers.
7	`docker compose start`	Starts services that were previously stopped.
8	`docker compose restart`	Restarts all containers defined in the compose file.
9	`docker compose down`	Stops and removes all containers and the default network, but keeps named volumes.
10	`docker compose down -v`	Stops and removes all containers, networks, AND named volumes (deletes database data) — run this last.

CONCLUSION

Through this experiment, a multi-service application consisting of a web server, a database, and a database administration tool was successfully deployed using Docker Compose. Instead of creating and managing each container individually, a single docker-compose.yml file was used to declaratively define all three services, their images, ports, environment variables, and persistent storage. The docker compose up -d command started all services together with a single instruction, and docker compose ps confirmed that all three containers were running and correctly networked. This experiment demonstrates the core advantage of Docker Compose over standalone Docker commands: it simplifies the deployment and orchestration of applications composed of multiple interdependent services, allows services to communicate using simple service names instead of IP addresses, and makes the entire application's configuration reproducible and version-controllable through a single file.